

PLAYWRIGHT NOTES

1. What is Playwright?

- Playwright is an **open-source automation testing framework**.
- It is developed by **Microsoft**.
- It is mainly used for **automating and testing web applications**.
- It supports major browser engines:
 - **Chromium - chrome , edge**
 - **Firefox**
 - **WebKit - safari**
- It supports multiple programming languages such as:
 - JavaScript / TypeScript
 - Java
 - Python
 - .NET
- Playwright can be used for **end-to-end testing** of web applications.

Interview Definition

Playwright is an open-source end-to-end test automation framework developed by Microsoft, used to automate and test web applications across Chromium, Firefox, and WebKit browsers.

2. Why Do We Use Playwright?

We use Playwright to:

- Automate web application testing.
- Reduce **manual testing effort**.
- Test applications across different browsers.
- Execute tests faster.
- Test complete **end-to-end user workflows**.
- Handle modern and dynamic web applications.
- Run multiple tests in parallel.
- Identify defects early.
- Improve test coverage.
- Reduce repetitive testing work.

Note : Flaky scripts : scripts might pass sometime and it might fail sometime(selenium cannot deal with dynamic elements(DOM Structure changes everytime))

Interview Answer

We use Playwright to automate web application testing, reduce manual effort, perform cross-browser testing, improve test coverage, and execute repetitive and end-to-end test scenarios efficiently.

3. Advantages of Playwright

1. Cross-Browser Support

- Supports **Chromium, Firefox, and WebKit**.
- The same test can be executed across different browsers.

2. Auto-Waiting → most import (30 sec)

- Playwright automatically waits for elements to become ready before performing actions.
- Reduces synchronization issues(speed of execution and speed of the web app) and flaky tests.

3. Fast Execution

- Playwright is designed for fast and efficient test execution.

4. Parallel Execution - imp

- Multiple tests can run simultaneously.
- Reduces overall execution time.

5. Easy Browser Management

- Supports browser, browser context, and page management.
- Allows us to create isolated browser sessions.

6. Multiple Tab/Window Handling - imp

- Can easily handle multiple tabs, windows, and popups.

7. Powerful Debugging

- Provides tools such as:
 - Playwright Inspector
 - Trace Viewer
 - Screenshots
 - Videos
 - HTML reports

8. Supports Modern Web Applications

- Works well with dynamic and modern web applications.

9. Open Source

- Playwright is freely available and has an active developer community.
-

4. Features of Playwright

Important features students should know:

- Cross-browser testing
 - Auto-waiting
 - Parallel test execution
 - Multiple tab and window handling
 - Iframe/Frame handling
 - Popup handling
 - Dialog handling
 - Mouse and keyboard actions
 - File upload and download
 - Screenshots and video recordings
 - Assertions
 - Hooks
 - Annotations
 - Test retries
 - Test timeout management - 30 sec
 - Trace Viewer
 - HTML reporting
-

5. Playwright Test Runner → execution takes place `@playwright/test`

Playwright provides its own **built-in test runner**, called **Playwright Test**.

It provides features such as:

- Test execution
- Assertions
- Fixtures
- Hooks
- Parallel execution
- Annotations
- Reporting

Interview Point

Playwright Test is the built-in test runner provided by Playwright for creating, executing, managing, and reporting automated tests.

6. Playwright – Quick Interview Revision

What is Playwright?

An open-source end-to-end test automation framework developed by Microsoft for testing web applications, api testing, mobile automation.

Why do we use Playwright?

To automate web application testing, reduce manual effort, perform cross-browser testing, and execute tests efficiently.

Which browsers does Playwright support?

Chromium(chrome, edge) , Firefox, and WebKit(safari).

Which languages does Playwright support?

JavaScript, TypeScript, Python, Java, and .NET.

Is Playwright open source?

Yes, Playwright is an open-source framework developed by Microsoft.

What is Playwright's biggest advantage?

It provides reliable cross-browser automation with features such as auto-waiting, parallel execution, browser contexts, powerful debugging.

Installation:

1. **Install Node.js** – Playwright requires Node.js and npm.

Why required? Playwright's JavaScript/TypeScript tooling runs on **Node.js**.

Use: Provides the **runtime environment** to execute Playwright tests.

What is it? npm = **Node Package Manager**.

Why required? Used to **install and manage Playwright and other project dependencies**.

Example: `npm init playwright@latest`

2. **Install Visual Studio** – For writing test scripts
3. **Check installation** – Verify using `node -v` and `npm -v`.
4. **Create project folder** – Create and open the project in VS Code.
5. **Open Terminal** – Open the terminal inside the project folder.
6. **Initialize Playwright** – Run `npm init playwright@latest`.
(if any error: `npm.cmd init playwright@latest`)
7. **Select language** – Choose **JavaScript** or **TypeScript**.

Playwright folder structure:

Playwright Project

```
|
|— tests/
|   |— example.spec.js
|
|— playwright.config.js
|— package.json
|— package-lock.json
|— node_modules/
```

1. `tests/`

- Main folder where we **store our test cases**.
- Usually contains files ending with `.spec.js` or `.test.js`.

Example: [login.spec.js](#) / [login.spec.ts](#)

2. playwright.config.js

- The **main configuration file** of Playwright.
 - Used to configure things like:
 - Browsers
 - Base URL
 - Timeout
 - Retries
 - Workers
 - Reports
 - Screenshots/videos
 - Projects
-

3. package.json

- Contains the **project information and dependencies**.
 - Stores Playwright and other npm package details.
 - Also contains scripts used to run the project.
-

4. package-lock.json

- Keeps the **exact versions of installed npm packages and their dependencies**.
 - Helps maintain consistent installations across environments.
-

5. node_modules/

- Contains the **installed packages/dependencies** required by the project.
- Created automatically when npm installs packages.

Important: We generally don't manually modify this folder.

Interview Point:

The Playwright project structure mainly consists of the tests folder for test cases, playwright.config.js for configuration, package.json for dependencies and project scripts, node_modules for installed packages, and report/result folders for test execution artifacts.

First code :

```
import {test} from "@playwright/test"

test("first code" , ()=>{

    console.log("hello");

})
```

Fixtures:

Fixtures are configurations or environments provided by Playwright that provide the required resources for executing a test.

Types of fixtures:

1. **Browser fixture** : Browser fixture is a pre-configured environment provided by Playwright that gives us a browser instance to run our tests.
2. **Context fixture** : Context fixture is a pre-configured environment provided by Playwright that gives us a browser context to run tests in an isolated session, where incognito mode is the default.
3. **Page fixture** : Page fixture is a pre-configured environment provided by Playwright that gives us a tab (page) to interact with and perform actions on a web application.
4. **browserName fixture** : Browser name fixture provides the name of the browser in which the test is being executed, such as Chromium, Firefox, or WebKit.
5. **Request** : This fixture is used in API automation

```
import {test} from "@playwright/test"

//browser fixture

test("fixtures", async({browser})=>{ //tells the browser

    let context = await browser.newContext() //browser session(isolated
session(profile), incognito mode)

    let page = await context.newPage() //tab within the browser session

    await page.goto("https://www.amazon.in/")

}) //control to multiple context(browser session) multiple windows
```

```

//context

test ("context fixture", async({context})=>{    //default it will create browser
session in the all the 3 browser

    let page = await context.newPage()

    await page.goto("https://www.amazon.in/")

}) //control over multiple tabs within one session


//page

test("page fixture", async({page})=>{    //tab inside the default 3 browsers

    await page.goto("https://www.amazon.in/")

}) //control over a single tab


//browsername

test("browsername fixture", async({browserName})=>{

    console.log(browserName);

})//return the name of the browser which is in used

```

Execute the test file in headed mode:

npx playwright test tests/[example.spec.js](#) --headed

Execute the test file in headless mode:

npx playwright test tests/[example.spec.js](#)

Annotation

Annotation is a special instruction used to provide additional information or control the behavior of a test case during execution.

Playwright Annotations

1. `test.skip()`

Definition: Used to **skip the execution of a test case**.

Syntax:

```
test.skip();
```

2. `test.only()`

Definition: Used to **execute only the selected test case** and skip all other tests.

Syntax:

```
test.only("test name", async ({ page }) => {  
    // test steps  
});
```

3. `test.fail()`

Definition: Used to **mark a test as expected to fail**.

Syntax:

```
test.fail();
```

4. `test.fixme()`

Definition: Used to **mark a test that needs to be fixed and skip its execution**.

Syntax:

```
test.fixme();
```

5. `test.slow()`

Definition: Used to **increase the timeout of a test that is expected to take more time**.

Syntax:

```
test.slow();
```

6. `test.describe()`

Definition: Used to **group related test cases together**.

Syntax:

```
test.describe("group name", () => {  
  
    // test cases  
  
});
```

Commands to run tests:

1. Run all the tests:-
npx playwright test
2. Run a specific test file:-
npx playwright test tests/[example.spec.js](#)
3. Run all the tests in headed mode:-
npx playwright test -- headed
4. Run a specific test file in headed mode:-
npx playwright test tests/[example.spec.js](#) -- headed
5. Run only the last failed test:-
npx playwright test -- last -- failed
npx playwright test -- last -- failed -- headed
6. Run a test with specific title:-
npx playwright test -g "your expected title"
npx playwright test tests/[example.spec.js](#) -g "your expected title"
7. Run all the test only in one browser:-
npx playwright test -- project= chromium
npx playwright test -- project= firefox
npx playwright test -- project= webkit
8. Run a specific test in one browser:-
npx playwright test tests/example.spec.js -- project= chromium
npx playwright test tests/[example.spec.js](#) -- project= firefox
npx playwright test tests/[example.spec.js](#) -- project= webkit
9. Run all the test in multiple browsers:-
npx playwright test -- project= chromium -- project= webkit

10. Run a specific test in multiple browser:-

```
npx playwright test tests/example.spec.js -- project= chromium -- project= webkit
```

11. Debug test in inspector mode:-

```
npx playwright test --debug - -headed
```

12. To generate report:-

```
npx playwright show-report
```

13. To generate report on specific port no:-

```
npx playwright show-report - -port = 9999
```

Browser Controls & Page Methods

These are some methods which are used to control the browser during the automation.

Example: minimize, maximize, navigation etc

1. `page.goto()` is used to navigate the current page to a specified URL.

Syntax :

```
await page.goto("URL");
```

2. `page.setViewportSize()` is used to **set or change the width and height of the page's viewport.**

Syntax:

```
await page.setViewportSize({  
  width: number,  
  height: number })
```

3. `page.viewportSize()` is used to **get the current width and height of the page's viewport.**

Syntax:

```
let size = page.viewportSize();  
console.log(size);
```

-
4. `page.title()` is used to **get the title of the current web page**.

Syntax:

```
let title = await page.title();
```

5. `page.url()` is used to **get the current URL of the page**.

Syntax:

```
let url = await page.url();
```

6. `context.cookies()` is used to **retrieve the cookies stored in the current browser context**.

Syntax:

```
let cookies = await context.cookies();
```

7. `chromium.launch()` is used to **launch a new Chromium browser instance**.

Syntax:

```
let browser = await chromium.launch();
```

8. `browser.newContext()` is used to **create a new isolated browser context (browser session) inside the browser instance**.

Syntax:

```
let context = await browser.newContext();
```

9. `context.newPage()` is used to **create a new page or tab inside the browser context**.

Syntax:

```
let page = await context.newPage();
```

10. `page.screenshot()` is used to **capture a screenshot of the current page**.

Syntax:

```
await page.screenshot({  
  path: "foldername/screenshotname.png/jep"  
});
```

11. `browser.close()` is used to **close the browser instance along with all its contexts and pages**.

Syntax:

```
await browser.close();
```

Examples on browser controls:

```
import {chromium, firefox, test, webkit} from  
"@playwright/test"  
  
//page.goto() navigate to a url  
  
test("page.goto()", async ({page}) => {  
  await page.goto("https://www.amazon.in/")  
})  
  
//page.setViewportSize() -- resize the browser  
  
test("page.setViewportSize", async ({page}) => {
```

```
    await page.setViewportSize({width:600 , height:
50000})

    // await page.waitForTimeout(4000)

  })

  //page.viewportsize() -- return the size of the browser
test("size of browser", async ({page}) => {

  let size = await page.viewportSize()

  console.log(size);

})

  //page.title() -- get the title of the current page
test("get title", async ({page}) => {

  await page.goto("https://www.amazon.in/")

  await page.waitForTimeout(4000)

  let title = await page.title()

  console.log(title);

  // console.log(page.title);

})

  //page.url() -- get the url of the current
test("get the url", async ({page}) => {

  await page.goto("https://www.amazon.in/")

  await page.waitForTimeout(4000)

  let url = await page.url()

  console.log(url);

})

  //page.screenshot()
```

```
test("screenshot", async ({page}) => {

    await page.goto("https://www.amazon.in/")

    await page.waitForTimeout(3000)

    await page.screenshot({path: "screenshot/ss1.png"})

})

//context.cookies() -- retrieve all cookies from the
current context(browser session) in an array

test("get context cookies" , async ({context}) => {

    let page = await context.newPage()

    await page.goto("https://www.amazon.in/")

    await page.waitForTimeout(3000)

    let cookies = await context.cookies()

    console.log(cookies);

})

//chromium.launch()--launches the chromium browser
instance

test("browser launch", async () => {

    let browser = await chromium.launch()

    // let browser = await webkit.launch()

    // let browser = await firefox.launch()

    let context = await browser.newContext()

    let page = await context.newPage()

    await page.goto("https://www.amazon.in/")

})
```

```

//browser.newcontext() -- used to create a browser
session

test("new context", async({browser})=>{

    let context = await browser.newContext()

    //context.page() -- used to create a tab/page withinh the
context

    let page = await context.newPage()

    await page.goto("https://www.amazon.in/")

}))

//browser.close()--close the browser instance

test("close browser", async({browser})=>{

    let context = await browser.newContext()

    let page = await context.newPage()

    await page.goto("https://www.amazon.in/")

    await browser.close()

}))

```

Locators in Playwright

Locators are used to **identify and locate web elements on a web page** so that Playwright can perform actions on them.

Examples of web elements:

- Text box
- Button
- Link
- Checkbox
- Radio button
- Dropdown

Types of Locators

The commonly used locator approaches are:

1. Locator methods → `page.locator(#locator)`
 - a. CSS selectors
 - b. Xpath
2. `getBy` methods

CSS Selector

A **CSS selector** is used to identify a web element based on its HTML attributes, ID, class, tag name, or a combination of these.

Selector Type	Syntax	Example
Class	<code>.classValue</code>	<code>.loginbtn</code>
ID	<code>#idValue</code>	<code>#submitbtn</code>
Tag + Class	<code>tag.classValue</code>	<code>button.loginbtn</code>
Tag + ID	<code>tag#idValue</code>	<code>input#username</code>
Attribute	<code>[attribute="value"]</code>	<code>[name="email"]</code>
Tag + Attribute	<code>tag[attribute="value"]</code>	<code>input[name="email"]</code>

XPath

XPath (XML Path Language) is used to identify and locate web elements in a web page by navigating through the HTML structure and using attributes, text, or other properties of the elements.

Types of XPath

XPath Type	Definition	Syntax	Example
Attribute XPath	Used to locate an element based on its attribute and attribute value .	<code>//tagName[@attribute="value"]</code>	<code>//input[@id="username"]</code>
Text XPath using <code>text()</code>	Used to locate an element based on its visible text using the <code>text()</code> function.	<code>//tagName[text()="text"]</code>	<code>//button[text()="Login"]</code>
Text XPath using <code>.</code>	Used to locate an element based on its text content using the <code>.</code> operator.	<code>//tagName[.="text"]</code>	<code>//button[.="Login"]</code>

GetBy Methods in Playwright

GetBy methods are Playwright's user-facing locators used to identify web elements based on how users interact with or see those elements.

They are generally more readable and maintainable than complex CSS or XPath selectors.

1. `getByRole()`

`getByRole()` is used to locate an element based on its ARIA role, such as button, textbox, link, checkbox, heading, etc.

Syntax

```
page.getByRole("role");
```

Example

```
await page.getByRole("button", { name: "Login" }).click();
```

Here, `"button"` is the role and `"Login"` is the accessible name.

2. `getByText()`

`getByText()` is used to locate an element based on the visible text displayed on the page.

Syntax

```
page.getByText("text");
```

Example

```
await page.getByText("Welcome to Amazon").click();
```

3. `getByLabel()`

`getByLabel()` is used to locate form elements using their associated label text.

It is commonly used with input fields such as username, email, password, etc.

Syntax

```
page.getByLabel("label text");
```

Example

```
await page.getByLabel("Username").fill("student");
```

4. `getByPlaceholder()`

`getByPlaceholder()` is used to locate an input element using its placeholder text.

Syntax

```
page.getByPlaceholder("placeholder text");
```

Example

```
await page.getByPlaceholder("Enter your email").fill("student@gmail.com");
```

5. `getByAltText()`

`getByAltText()` is used to locate an image or other element using its `alt` attribute value.

Syntax

```
page.getByAltText("alt text");
```

Example

```
await page.getByAltText("Amazon logo").click();
```

HTML:

```

```

6. `getByTitle()`

`getByTitle()` is used to locate an element using the value of its `title` attribute.

Syntax

```
page.getByTitle("title text");
```

Example

```
await page.getByTitle("Close").click();
```

HTML:

```
<button title="Close">X</button>
```

7. `getByTestId()`

`getByTestId()` is used to locate an element using a specific test ID assigned to the element.

By default, Playwright looks for the `data-testid` attribute.

Syntax

```
page.getByTestId("test-id");
```

Example

```
await page.getByTestId("login-button").click();
```

HTML:

```
<button data-testid="login-button">Login</button>
```

Example:

```
import {test} from "@playwright/test"

test("login" , async ({page}) => {

    await
page.goto("https://practicetestautomation.com/practice-test-1
login/")

    await
page.locator('input[id="username"]').fill("student")

    await
page.locator('input[id="password"]').fill("Password123")

    await page.locator('button[id="submit"]').click()

})
```

Interview Question:

What are locators in Playwright, and what is the difference between `page.locator()` and `getBy` methods? Give examples of both.

Answer:

Locators are used to identify and interact with web elements on a web page.

`page.locator()` is commonly used with CSS selectors or XPath to locate an element.

```
await page.locator("#username").fill("student");
```

`getBy` methods are user-facing locators that identify elements based on how users interact with or see them, such as role, text, label, or placeholder.

```
await page.getByRole("button", { name: "Login" }).click();
```

Element Controls in Playwright

Element controls are Playwright methods used to **interact with, retrieve information from, and check the state of web elements**.

1. `fill()`

`fill()` is used to enter text into an input element. It **clears the existing value first** and then enters the new text.

Syntax

```
await page.locator(".class").fill("text");
```

Example

```
await page.locator("#username").fill("admin");
```

If `"user"` was already present, `fill("admin")` replaces it with `"admin"`.

2. `type()`

`type()` is used to type text into an element **character by character**. It can be useful when you want to simulate typing with a delay between characters.

Syntax

```
await page.locator(".class").type("text", { delay: 500 });
```

Example

```
await page.locator("#username").type("admin", { delay: 500 });
```

Here, `500` means Playwright waits **500 milliseconds between each character**.

fill() vs **type()**

fill() → enters the complete text and replaces existing text

type() → types character by character

3. click()

click() is used to perform a click action on a web element.

It can be used with:

- Buttons
- Links
- Checkboxes
- Radio buttons
- Dropdown options
- Other clickable elements

Syntax

```
await page.locator(".class").click();
```

Example

```
await page.locator("#submit").click();
```

4. innerText()

innerText() returns the **visible text** of an element as it would appear to the user.

It is generally used with a **single element**.

Syntax

```
const text = await page.locator(".class").innerText();
```

Example

HTML:

```
<div>
```

```
  "Hello"
```

```
  <span style="display:none">"world"</span>
```

```
</div>
```

```
const text = await page.locator("div").innerText();
```

```
console.log(text);
```

Output:

Hello

The hidden "World" is not included.

5. `textContent()`

`textContent()` returns the **text content inside an element**, including text from hidden child elements.

It is generally used with a **single element**.

Syntax

```
const text = await page.locator(".class").textContent();
```

Using the same HTML:

```
<div>
```

```
  Hello
```

```
  <span style="display:none">World</span>
```

```
</div>
```

```
const text = await page.locator("div").textContent();
```

```
console.log(text);
```

The result contains both:

Hello

World

Easy Difference

`innerText()` → visible text

`textContent()` → text content, including hidden text

6. `allTextContents()`

`allTextContents()` retrieves the text content of **all matching elements** and returns the results as an **array of strings**.

Syntax

```
const texts = await page.locator(".class").allTextContents();
```

Example

Suppose the page contains:

```
<div class="name">Tom</div>
```

```
<div class="name">Jerry</div>
```

```
<div class="name">Sam</div>
```

Code:

```
const names = await page.locator(".name").allTextContents();
```

```
console.log(names);
```

Output:

```
["Tom", "Jerry", "Sam"]
```

7. `getAttribute()`

`getAttribute()` retrieves the **value of a specific HTML attribute** from an element.

Syntax

```
const value = await page.locator(".class").getAttribute("attribute_name");
```

Example

HTML:

```
<input id="FirstName" name="firstname">
```

Code:

```
const value = await page.locator("#FirstName").getAttribute("name");
```

```
console.log(value);
```

Output:

```
firstname
```

8. **all()**

all() returns **all matching elements as an array of Locator objects**.

Syntax

```
const elements = await page.locator(".class").all();
```

Example

```
const buttons = await page.locator("button").all();  
  
console.log(buttons);
```

9. **selectOption()**

selectOption() is used to **select an option from a `<select>` dropdown**.

Syntax

```
await page.locator("select").selectOption("option_value");
```

10. **isVisible()**

isVisible() checks whether an element is **visible on the page**.

It returns a **boolean value**:

true → element is visible

false → element is not visible

Syntax

```
const result = await page.locator(".class").isVisible();
```

Example

```
const visible = await page.locator("#login").isVisible();  
  
console.log(visible);
```

Output:

```
true
```

11. isEnabled()

`isEnabled()` checks whether an element is **enabled and can be interacted with**.

It returns:

true → element is enabled

false → element is disabled

Syntax

```
const result = await page.locator(".class").isEnabled();
```

Example

```
const enabled = await page.locator("#submit").isEnabled();
```

```
console.log(enabled);
```

12. isChecked()

`isChecked()` checks whether a **checkbox or radio button** is currently selected/checked.

It returns:

true → checked

false → unchecked

Syntax

```
const result = await page.locator(".class").isChecked();
```

Example

```
const checked = await page.locator("#terms").isChecked();
```

```
console.log(checked);
```

Mouse Actions in Playwright

Mouse actions are used to **simulate mouse interactions** with web elements or specific coordinates on a web page.

1. Click:

`click()` performs a click action on a web element.

By default, Playwright performs a **left-click**.

Syntax

```
await page.locator("locator").click();
```

Or explicitly:

```
await page.locator("locator").click({ button: "left" });
```

Right Click

Used to perform a **right-click** on an element.

```
await page.locator("locator").click({ button: "right" });
```

Middle Click

Used to perform a **middle-click** on an element.

```
await page.locator("locator").click({ button: "middle" });
```

2. Double Click:

`dblclick()` performs a **double-click** action on a web element.

Syntax

```
await page.locator("locator").dblclick();
```

You can also perform two clicks using `clickCount`:

```
await page.locator("locator").click({ clickCount: 2 });
```

3. Mouse.Move:

`mouse.move()` moves the mouse pointer to a **specific coordinate** on the web page.

Syntax

```
await page.mouse.move(x, y);
```

Example

```
await page.mouse.move(500, 300);
```

Here:

- 500 → X-coordinate
- 300 → Y-coordinate

Note: The correct syntax is `page.mouse.move()`, not `page.mouse.move()`.

4. Mouse.Down:

`mouse.down()` presses and holds the mouse button without releasing it.

Syntax

```
await page.mouse.down();
```

It is commonly used with `mouse.move()` for actions such as **dragging**.

Example:

```
await page.mouse.move(200, 200);
```

```
await page.mouse.down();
```

```
await page.mouse.move(500, 300);
```

```
await page.mouse.up();
```

5. Mouse.Up:

`mouse.up()` releases the mouse button after it has been pressed using `mouse.down()`.

Syntax

```
await page.mouse.up();
```

Usually used together with:

```
await page.mouse.down();
```

```
await page.mouse.up();
```

6. Hover:

`hover()` moves the mouse pointer over a web element **without clicking it**.

It is commonly used for:

- Displaying dropdown menus
- Showing tooltips
- Revealing hidden elements
- Mouse-over menus

Syntax

```
await page.locator("locator").hover();
```

Example

```
await page.getByText("Products").hover();
```

7. Dispatch Event:

`dispatchEvent()` triggers a **browser event directly on an element** without physically performing the corresponding mouse action.

Syntax

```
await page.locator("locator").dispatchEvent("click");
```

Example

```
await page.locator("#submit").dispatchEvent("click");
```

Here, Playwright directly triggers the `click` event on the element.

```
import {test} from "@playwright/test"
//left click
test("click action", async ({page}) => {
  await page.goto('https://demoapps.qspiders.com/ui/button?sublist=0')
  await page.locator('//button[text()='Yes"]').click({button: "left"}) //click on
yes button
  await page.waitForTimeout(3000)
  await page.locator('//button[text()='No"]').click() //click on no button
  await page.waitForTimeout(3000)
})
//right click
```

```

test("right click", async ({page}) => {
  await
page.goto('https://demoapps.qspiders.com/ui/button/buttonRight?sublist=1')
  await page.locator('button[id="btn_a"]').click({button: 'right'})
  await page.waitForTimeout(3000)
  await page.locator('//div[text()="Yes"]').click() //yes button
  await page.waitForTimeout(3000)
  await page.locator('button[id="btn_a"]').click({button: 'right'})
  await page.locator('//div[text()="No"]').click() //no button
  await page.waitForTimeout(3000)
})

//double click
test('double click', async ({page}) => {
  await
page.goto('https://demoapps.qspiders.com/ui/button/buttonDouble?sublist=2')
  await page.locator('button[id="btn_a"]').dblclick() //yes button
  await page.waitForTimeout(3000)
  await page.locator('button[id="btn_b"]').click({clickCount: 2}) //click on No
Button
  await page.waitForTimeout(3000)
})

//hover
test('hover action', async ({page}) => {
  await page.goto('https://demoapps.qspiders.com/ui/mouseHover?sublist=0')
  await page.locator('img[src="/assets/message-hint-nbRmWGWf.png"]').hover()
  await page.waitForTimeout(3000)
})

//mouse down and mouse up
test('mouse up and down', async ({page}) => {
  await page.goto('https://demoapps.qspiders.com/ui/clickHold?sublist=0')
  await page.locator('div[id="circle"]').hover()
  await page.waitForTimeout(3000)
  await page.mouse.down() //pressing without releasing
  await page.waitForTimeout(4000)
  await page.mouse.up() //releasing the mouse.down action
  await page.waitForTimeout(3000)
})

//move action
test
("move action", async ({page}) => {
  await page.goto('https://demoapps.qspiders.com/ui/dragDrop?sublist=0')
  await page.locator('//div[text()="Drag Me"]').hover()
  await page.waitForTimeout(3000)
  await page.mouse.down()

```

```
    await page.waitForTimeout(3000)
    await page.mouse.move(400, 500)
    await page.waitForTimeout(3000)
    await page.mouse.up()
    await page.waitForTimeout(3000)
  })

  //dispatch event
  test('dispatch event', async ({page}) => {
    await
page.goto('https://demoapps.qspiders.com/ui/button/buttonDisabled?sublist=4')
    await page.getByText("submit").first().dispatchEvent("click")
    await page.getByText("submit").first().click({force: true})
    await page.waitForTimeout(3000)
  })
```

Waits in Playwright

Wait is a mechanism that pauses the execution of the test script until a specific condition is met or a defined timeout period passes.

Why Are Waits Necessary?

1. The test script might try to click on a button before it becomes clickable.
2. The test script might attempt to read text from an element that hasn't been loaded yet.
3. Test script might fail due to transitions, animations, or slow network response.

Waits help to:

- Reduce flakiness in the test.
- Ensure consistency across different environments.
- Improve reliability.
- Reflect actual user behaviour.

Types of Waits in Playwright

Playwright provides the following types of waits:

1. Auto Waiting

- Smart built-in waits

2. Hard-Coded Waits

- Static waits

3. Explicit Waits

3.1 Element-Based Waits

- Wait for text
- Wait for timeout
- Wait for selector
- Wait for element state

3.2 Page-Level Waits

- Navigation wait
- Load state wait
- Wait for event

3.3 Custom Waits

1. Auto Waiting

- Auto waiting is a **built-in Playwright feature**.
- It waits automatically for an element to be ready before performing an action.
- Common actions that support auto-waiting:

click()
fill()
type()
check()
hover()
dblclick()
uncheck()

- Playwright automatically waits for up to **30 seconds** by default.

1. Page-Level Timeout

```
await page.setDefaultTimeout(40000);
```

- Sets the **default action timeout** to 40 seconds for that page.
- Applies to actions such as `click()`, `fill()`, `check()`, `hover()`, etc.

2. Individual Action Timeout

```
await page.locator("#login").click({ timeout: 40000 });
```

- Sets the timeout to **40 seconds for that particular action only**.
- Other actions will continue using their existing timeout.

3. Test-Level Action Timeout

```
test.use({ actionTimeout: 50000 });
```

- Sets the **default action timeout to 50 seconds for the test.**
- Applies to Playwright actions performed within that test.

```
//all the action in the script

test.use({actionTimeOut:50000})

test("right click", async ({page}) => {

    //change the default timeout

    await page.setDefaultTimeout(40000)

    await
page.goto("https://demoapps.qspiders.com/ui/button/buttonRight?sublist=
1")

    await page.locator('button[id="btn_a"]').click({button: "right"})

    //for individual action

    await page.locator('//div[text()="No"]').click({timeout:40000})
//30sec

    await page.waitForTimeout(3000)

})
```

2. Hard-Coded Waits / Static Waits

- A hard-coded wait is a **fixed delay** where Playwright pauses the execution for a specific amount of time.
- It does not wait for a specific condition.

Syntax

```
await page.waitForTimeout(4000);
```

Here, Playwright pauses the execution for **4000 milliseconds (4 seconds)**.

Disadvantages

- Increases the execution time.
- Makes the test slower.
- Not recommended for normal test execution.
- Cannot cope with the actual speed of the application.

When to Use

Hard-coded waits can be used for:

- Debugging the test.
- Troubleshooting timing issues.

```
// await
page.locator('input[id="submit"]').dispatchEvent("click")

await page.locator('input[id="submit"]').click()

await page.waitForTimeout(3000)
```

3. Explicit Waits in Playwright

Explicit waits are useful when the application performs an action dynamically and we need to wait for a particular condition.

Explicit Waits Can Be Used For

- Waiting for an element to be **present, visible, hidden, or detached**.
- Waiting for a specific **text or data** to appear.
- Waiting for **page navigation** to complete.
- Waiting for a particular **page load state**.
- Waiting for a specific **event** to occur.
- Waiting for a custom condition to become true.

Types of Element-Based Waits:

1. Wait for Text

Used to wait until a specific text appears inside an element.

2. Wait for Timeout

Used to pause the test execution for a specific amount of time.

3. Wait for Selector

Used to wait for an element matching a CSS/XPath selector to reach a specific state.

4. Wait for Element State

Used to wait for a locator to reach a specific state.

Common states:

visible → Waits until the element is **present in the DOM and visible** to the user.

attached → Waits until the element is **present/attached to the DOM**.

hidden → Waits until the element is **hidden or not visible**.

detached → Waits until the element is **removed from the DOM**.

```
import {test} from "@playwright/test"
//wait for text
test("wait for text", async ({page}) => {
  await page.goto("https://shoppersstack.com/")
  await page.locator('//h3[text()="Welcome to ShoppersStack. Enjoy shopping with us."],
    {hasText:"Welcome to ShoppersStack. Enjoy shopping with us."}).waitFor()

  await page.locator('//button[text()="Login"]', {hasText:'Login'}).waitFor()
})

//wait for timeout -- give waits for a specific element
test("wait for timeout", async ({page}) => {
  await page.goto("https://shoppersstack.com/")
  await page.locator('//button[text()="Login"]').waitFor({timeout:50000,
state:'attached'})
})

//wait for selector
test("wait for selector", async ({page}) => {
  await page.goto("https://shoppersstack.com/")
  await page.waitForSelector('//h3[text()="Welcome to ShoppersStack. Enjoy shopping with us."])
  await page.waitForSelector('//button[text()="Login"]')
})

//wait for state
test("wait for state", async ({page}) => {
  await page.goto("https://shoppersstack.com/")
  await page.locator('//button[text()="Login"]').waitFor({state:'attached'}) //30
sec
})
```

Types of Page Level Waits:

Wait for Navigation

`waitForNavigation()` is used to wait until a page navigation is completed before continuing the test execution.

Syntax

`await page.waitForNavigation();`

```
//wait for navigation
test.only('wait for navigation', async ({browser}) => {
  let context = await browser.newContext()
  let page = await context.newPage()
  await page.goto('https://demoapps.qspiders.com/ui/browser?sublist=0')
  let [nav] = await Promise.all([
    page.waitForNavigation(),
    page.locator('//button[text()="view more"]').first().click()
  ])
})
})
```

Wait for Load State

`waitForLoadState()` is used to wait until the page reaches a specific loading state before continuing the test execution.

Syntax

`await page.waitForLoadState("load");`

Example

```
test("wait for load state", async ({ page }) => {

  await page.goto("https://www.amazon.in/");

  await page.waitForLoadState("networkidle");

  console.log("loaded");

  await page.getByPlaceholder("Search Amazon.in").fill("shirt");
});
```

Load States

1. `load`

Waits until the `load` event is fired and the page's resources such as images and stylesheets have finished loading.

`await page.waitForLoadState("load");`

2. domcontentloaded

Waits until the HTML document has been completely loaded and parsed, without waiting for all resources such as images to finish loading.

```
await page.waitForLoadState("domcontentloaded");
```

3. networkidle

Waits until there are no active network connections(api) for a short period of time.

```
await page.waitForLoadState("networkidle");
```

Wait for Event

`waitForEvent()` is used to wait for a **specific browser event** before continuing the test execution.

Syntax

```
await page.waitForEvent("event");
```

Example

```
const popup = await page.waitForEvent("popup");
```

Here, Playwright waits until a **popup/new tab** is opened.

Common Events

- `popup` → waits for a new popup or tab
- `page` → waits for a new page
- `download` → waits for a file download
- `dialog` → waits for an alert, confirm, or prompt
- `filechooser` → waits for a file chooser

Important Point

`waitForEvent()` should be written **before the action that triggers the event** to avoid a race condition.

4. Custom Wait

Custom wait is used when the required waiting condition is specific to the application and cannot be handled directly by Playwright's standard waits.

`waitForFunction()`

`waitForFunction()` waits until the given **JavaScript condition returns `true`**.

Syntax

```
await page.waitForFunction(() => {  
    return condition;  
});
```

Example

```
await page.waitForFunction(() => {  
    return document.readyState === "interactive";  
});
```

Here, Playwright continues the test only when `document.readyState` becomes **interactive**.

Important Point

`waitForFunction()` should be used only when Playwright's built-in waits and assertions cannot handle the required condition.

Scroll Action

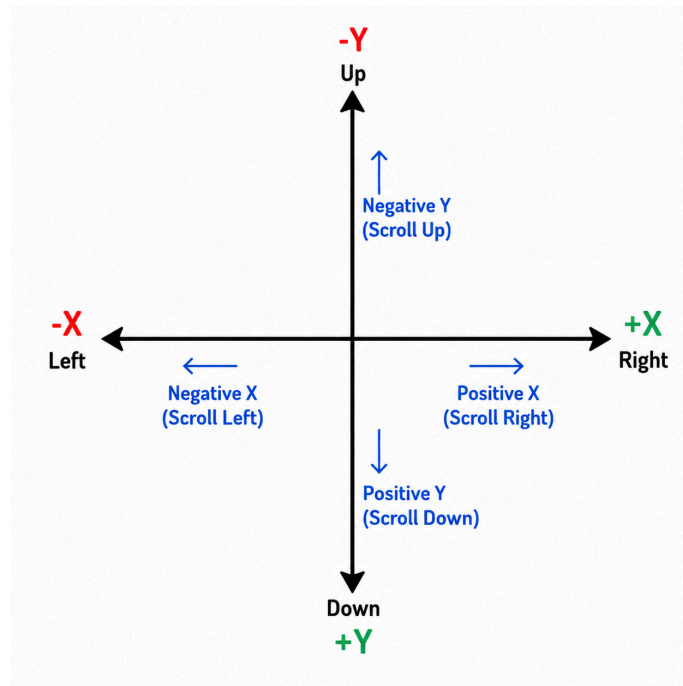
Scrolling means moving the webpage content up, down, left, or right to view content that is not currently visible on the screen.

In Playwright, we can perform scrolling using:

```
await page.mouse.wheel(x, y);
```

X and Y Axis

- **X-axis** → controls **horizontal scrolling** (left and right)
- **Y-axis** → controls **vertical scrolling** (up and down)



Positive and Negative Values

Y-axis – Vertical Scroll

await page.mouse.wheel(0, 500);

+Y → Scrolls **down**

await page.mouse.wheel(0, -500);

-Y → Scrolls **up**

X-axis – Horizontal Scroll

await page.mouse.wheel(500, 0);

+X → Scrolls **right**

await page.mouse.wheel(-500, 0);

-X → Scrolls **left**

```
import {test} from "@playwright/test"

//normal vertical scroll

test("vertical scroll" , async ({page}) => {

    await page.goto('https://demoapps.qspiders.com/ui/scroll/newTabVertical')
```

```

    await page.waitForTimeout(3000)

    await page.mouse.wheel(0,2000)    //horizontal (x,0)        vertical(0,y)

    await page.waitForTimeout(3000)

    await page.mouse.wheel(0,-1000)

    await page.waitForTimeout(3000)

  })

  //scrolling vertically till an element

  test.only("scrolling till element" , async ({page}) => {

    await page.goto('https://demoapps.qspiders.com/ui/scroll/newTabVertical')

    await page.waitForTimeout(3000)

    await page.locator('input[type="checkbox"]').scrollIntoViewIfNeeded()

    await page.waitForTimeout(3000)

    await page.locator('input[type="checkbox"]').click()

    await page.waitForTimeout(3000)

    await page.getByText('Accept Our Policy').click()

    await page.waitForTimeout(3000)

  })

  //horizontal scroll

  test.only("horizontal scroll", async ({page}) => {

    await page.goto('https://demoapps.qspiders.com/ui/scroll/newTabHorizontal')

    await page.waitForTimeout(3000)

    await page.mouse.wheel(7000,0)    //towards right

    await page.waitForTimeout(3000)

    await page.mouse.wheel(-7000,0)   //towards left

    await page.waitForTimeout(3000)

  })

```

Drag and Drop

Drag and drop in Playwright is a mouse action used to click and hold an element, move it to another location, and release it.

It is commonly used to automate actions like moving a file, rearranging items, or dragging an object into a specific area.

```
import {test} from "@playwright/test"

//drag and drop using mouse actions:

test("drag and drop using mouse actions", async ({page}) => {

    await page.goto('https://demoapps.qspiders.com/ui/dragDrop?sublist=0')

    await page.locator('//div[text()='Drag Me"]').hover()

    await page.mouse.down()

    await page.waitForTimeout(3000)

    await page.mouse.move(400,300)

    await page.waitForTimeout(3000)

    await page.mouse.up()

    await page.waitForTimeout(3000)

})

//drag and drop to target using mouse action

test("drag and drop to target using mouse action", async ({page}) => {

    await
page.goto('https://demoapps.qspiders.com/ui/dragDrop/dragToCorrect?sublist=2')

    await page.locator('//div[text()='Mobile Charger"]').hover()

    await page.waitForTimeout(3000)

    await page.mouse.down()

    await page.waitForTimeout(3000)

    await page.locator('//div[text()='Mobile Accessories"]').hover()

    await page.waitForTimeout(3000)
```

```

    await page.mouse.up()

    await page.waitForTimeout(3000)

    await page.locator('//div[text()="Laptop Charger"]').hover()

    await page.waitForTimeout(3000)

    await page.mouse.down()

    await page.waitForTimeout(3000)

    await page.locator('//div[text()="Laptop Accessories"]').hover()

    await page.waitForTimeout(3000)

    await page.mouse.up()

    await page.waitForTimeout(3000)
  })

  //drag and drop to target using bounding box

  test.only("drag and drop to target using bounding box", async ({page}) => {

    await
page.goto('https://demoapps.qspiders.com/ui/dragDrop/dragToCorrect?sublist=2')

    await page.locator('//div[text()="Mobile Charger"]').hover()

    await page.waitForTimeout(3000)

    await page.mouse.down()

    let box = await page.locator('//div[text()="Mobile
Accessories"]').boundingBox()

    console.log(box);

    await page.mouse.move(box.x , box.y)

    await page.waitForTimeout(3000)

    await page.mouse.up()

    await page.waitForTimeout(3000)

    await page.locator('//div[text()="Laptop Charger"]').hover()

    await page.waitForTimeout(3000)
  })

```

```
    await page.mouse.down()

    await page.waitForTimeout(3000)

    let box1 = await page.locator('//div[text()="Laptop Accessories"]').boundingBox()

    await page.mouse.move(box1.x , box1.y)

    await page.waitForTimeout(3000)

    await page.mouse.up()

    await page.waitForTimeout(3000)
  })

  //using dragto()

  test.only("drag and drop using drag method", async ({page}) => {

    await
page.goto('https://demoapps.qspiders.com/ui/dragDrop/dragToCorrect?sublist=2')

    let source = await page.locator('//div[text()="Mobile Charger"]')

    let target = await page.locator('//div[text()="Mobile Accessories"]')

    await source.dragTo(target)

    await page.waitForTimeout(3000)
  })
}
```

Keyboard Actions

Keyboard actions in Playwright are used to **simulate keyboard operations** such as typing text, pressing keys, using shortcuts, and navigating through a webpage using the keyboard.

Playwright provides keyboard actions through the **page.keyboard** object.

Types of Keyboard actions:

1. **keyboard.type()**

Definition: Types text using keyboard actions.

Syntax:

```
await page.keyboard.type("text");
```

2. **keyboard.press()**

Definition: Presses a specific keyboard key.

Syntax:

```
await page.keyboard.press("Enter");
```

3. **keyboard.insertText()**

Definition: Inserts text directly into the focused element.

Syntax:

```
await page.keyboard.insertText("text");
```

4. **keyboard.down()**

Definition: Presses and holds a keyboard key.

Syntax:

```
await page.keyboard.down("Shift");
```

5. **keyboard.up()**

Definition: Releases a previously pressed keyboard key.

Syntax:

```
await page.keyboard.up("Shift");
```

```
import {test} from "@playwright/test"

test("type action", async ({page}) => {

    await page.goto('https://demoapps.qspiders.com/ui')

    //type

    await page.locator('input[id="name"]').type('shinchan')

    await page.waitForTimeout(3000)

    //or

    await page.locator('input[id="name"]').click()

    await page.keyboard.type('shinchan')

    await page.waitForTimeout(3000)

    //or

    await page.type('input[id="name"]', 'shinchan')

    await page.waitForTimeout(3000)

})
```

```
test('insert text', async ({page}) => {

    await page.goto('https://demoapps.qspiders.com/ui')

    //insert text

    await page.locator('input[id="name"]').click()

    await page.keyboard.insertText('shinchan')

    await page.waitForTimeout(3000)

    //press

    await page.keyboard.press('Space')

    await page.waitForTimeout(3000)

    //up and down action
```

```
await page.keyboard.down('N')

await page.keyboard.up('N')

await page.waitForTimeout(2000)

await page.keyboard.down('o')

await page.keyboard.up('o')

await page.waitForTimeout(2000)

await page.keyboard.down('h')

await page.keyboard.up('h')

await page.waitForTimeout(2000)

await page.keyboard.down('a')

await page.keyboard.up('a')

await page.waitForTimeout(2000)

await page.keyboard.down('r')

await page.keyboard.up('r')

await page.waitForTimeout(2000)

await page.keyboard.down('a')

await page.keyboard.up('a')

await page.waitForTimeout(2000)

//press

await page.keyboard.press('Tab')

await page.waitForTimeout(3000)

await page.locator('input[id="email"]').type('abc@gmail.com')

await page.waitForTimeout(3000)

await page.keyboard.press('Control+a')

await page.keyboard.press('Control+c')
```

```
await page.keyboard.press('Tab')

await page.keyboard.press('Control+v')

await page.waitForTimeout(2000)

await page.locator('//button[text()="Register"]').click()

}))
```

Checkbox

A checkbox is a UI element that allows the user to select or deselect one or multiple options.

Syntax:

```
await page.locator('selector').check();
```

check() – Selects/checks the checkbox.

unchecked() – Deselects/unchecks the checkbox.

Radio Button

A radio button is a UI element that allows the user to select **only one option from a group of options**.

Syntax:

```
await page.locator('selector').check();
```

check() – Selects the radio button.

Difference Between **check()** and **click()**

check()

- Used specifically for **checkboxes and radio buttons**.
- Selects the element only if it is not already selected.
- Ensures the element is in a checked state.

click()

- Used to **click any clickable element** such as buttons, links, checkboxes, radio buttons, etc.
- Simply performs a mouse click on the element.

```
import {test} from "@playwright/test"

//checkboxes

test("dealing with checkboxes", async ({page}) => {

    await page.goto('https://demoapps.qspiders.com/ui/checkbox?sublist=0')

    await page.waitForTimeout(3000)

    //click method

    await page.locator('input[id="domain_b"]').click()

    await page.waitForTimeout(3000)

    await page.locator('input[id="domain_b"]').click()

    await page.waitForTimeout(3000)

    //check()

    await page.locator('input[id="domain_a"]').check()

    await page.waitForTimeout(3000)

    await page.locator('input[id="domain_a"]').unchecked()

    await page.waitForTimeout(3000)

})

//radio button

test.only('dealing with radio buttons', async ({page}) => {

    await page.goto('https://demoapps.qspiders.com/ui/radio?sublist=0')

    await page.locator('input[value="Upi"]').check()

    await page.waitForTimeout(3000)

})
```

Dropdowns

A dropdown is a UI element that allows the user to **select an option from a list of available options**.

Types of Dropdowns in Playwright

1. Standard (Native) Dropdown
2. Multi-Select Dropdown
3. Custom Dropdown
4. Autosuggestion / Searchable Dropdown

Standard (Native) Dropdown

A standard dropdown is an HTML `<select>` element that allows the user to select **one option from a predefined list of options**.

Syntax

1. Select by Value

```
await page.locator('selector').selectOption({ value: 'value' });
```

Selects an option using its **value attribute**.

2. Select by Index

```
await page.locator('selector').selectOption({ index: index_value });
```

Selects an option based on its **index position** in the dropdown.

3. Select by Label

```
await page.locator('selector').selectOption({ label: 'label_value/visible_text' });
```

Selects an option using its **label/text displayed in the dropdown**.

4. Select by Visible Text

```
await page.locator('selector').selectOption('visible_text');
```

Selects an option using its **visible text**.

In short:

- **value** → selects using the **value** attribute
- **index** → selects using the option's position
- **label** → selects using the displayed label
- **visible text** → selects using the option text

```
import {test} from "@playwright/test"

//standard dropdown

test('standard dropdown', async ({page}) => {

  await page.goto('https://demoapps.qspiders.com/ui/dropdown')

  await page.waitForTimeout(3000)

  //selecting through value

  await page.locator('select[id="select3"]').selectOption({value:"India"})

  await page.waitForTimeout(3000)

  //selecting through label

  await page.locator('select[id="select5"]').selectOption({label: 'Gujarat'})

  await page.waitForTimeout(3000)

  //selecting through visible text

  await page.locator('select[id="select5"]').selectOption('Gujarat')

  await page.waitForTimeout(3000)


  //selecting through index

  await page.locator('select[class="border-0 px-3 py-3 placeholder-blueGray-300 text-blueGray-600 bg-white rounded text-sm shadow focus:outline-none focus:ring w-full ease-linear transition-all duration-150"]')

  .nth(2).selectOption({index:5})

  await page.waitForTimeout(3000)

})
```

Multi-Select Dropdown

A multi-select dropdown is a dropdown that allows the user to **select multiple options at the same time** from a list of available options.

Syntax

1. Select by Value

```
await page.locator('selector').selectOption( [{ value: 'value1' }, { value: 'value2' }, { value: 'value3' } ]);
```

Selects an option using its **value attribute**.

2. Select by Index

```
await page.locator('selector').selectOption( [{ index: index_value1 }, { index: index_value2 }, { index: index_value3 } ]);
```

Selects an option based on its **index position** in the dropdown.

3. Select by Label

```
await page.locator('selector').selectOption( [{ label: 'label_value1/visible_text' }, { label: 'label_value2/visible_text' }, { label: 'label_value3/visible_text' } ]);
```

Selects an option using its **label/text displayed in the dropdown**.

4. Select by Visible Text

```
await page.locator('selector').selectOption(['visible_text1' , 'visible_text2' , 'visible_text3' ]);
```

Selects an option using its **visible text**.

```
//multiselect dropdown

test('multiselect dropdown', async ({page}) => {

  await
page.goto('https://demoapps.qspiders.com/ui/dropdown/multiSelect?sublist=1')

  await page.locator('select[id="select-multiple-native"]')

    .selectOption([ {value:"Rain Jacket Women Windbreaker Striped Climbing Raincoats"},

      {value:"MBJ Women's Solid Short Sleeve Boat Neck V "},

    ])

  await page.waitForTimeout(3000)
```

```

    await page.locator('button[class="bg-orange-500 p-2 text-white rounded w-[150px]"') .click()

    await page.waitForTimeout(3000)
  })

```

Custom Dropdown

A **custom dropdown** is a dropdown that is created using normal HTML elements like `div`, `li`, `span`, etc., instead of the HTML `<select>` tag.

Because it is **not a `<select>` element**, we **cannot use `selectOption()`**.

```

//custom dropdowns
//1st way to handle using locator/xpath
test('custom dropdown', async ({page}) => {
  await
page.goto("https://www.amazon.in/s?k=shirt&crd=LFSNGFUUAXJ8&sprefix=shirt%2Caps%2C307&ref=nb_sb_noss_2")
  await page.waitForSelector('span[id="a-autoid-0-announce"]')
  await page.locator('span[id="a-autoid-0-announce"]').click()
  await page.waitForTimeout(3000)
  await page.locator('a[id="s-result-sort-select_4"]').click()
  await page.waitForTimeout(3000)
})

//2nd way to handle through iteration
test.only('custom dropdown through itr', async ({page}) => {
  await
page.goto("https://www.amazon.in/s?k=shirt&crd=LFSNGFUUAXJ8&sprefix=shirt%2Caps%2C307&ref=nb_sb_noss_2")
  await page.locator('span[id="a-autoid-0-announce"]').click()
  await page.waitForSelector('a[class="a-dropdown-link"]')
  let options = await page.locator('a[class="a-dropdown-link"]').all()
  for(let option of options)
  {
    let text = await option.textContent()
    if (text.includes('Newest'))
    {
      await option.click()
      break;
    }
  }
  await page.waitForTimeout(3000)
})

```

Auto Suggestions

Auto-suggestions are the options that appear automatically when we type something into a search box or input field.

```
//autosuggestions

test.only('autosuggestions', async ({page}) => {

  await page.goto('https://www.amazon.in/')

  await page.locator('input[id="twotabsearchtextbox"]').fill('shirt')

  await page.waitForSelector('div[class="s-suggestion-container"]')

  let autosuggs = await (await page.locator('div[class="s-suggestion
s-suggestion-ellipsis-direction"]')).all()

  console.log(autosuggs);

  for(let autosugg of autosuggs)

  {

    let text1 = await autosugg.textContent()

    if(text1.includes(' men'))

    {

      await autosugg.click()

      break;

    }

  }

  await page.waitForTimeout((3000))

})
```

New Tab and New Window

New Tab/New Window handling in Playwright is the process of handling a new **Page** that is opened when an action such as clicking a link or button opens another tab or browser window.

New Tabs:

```
import { test } from '@playwright/test';

test("open two pages", async ({ browser }) => {

  let context = await browser.newContext();

  let page1= await context.newPage()

  await
page1.goto('https://www.flipkart.com/search?q=laptop&otracker=search&otracker1=search&marketplace=FLIPKART&as-show=on&as=off')

  await page1.waitForTimeout(3000)

  await page1.locator('//div[text()="MOTOROLA Motobook 60 Full Metal 2.8K OLED (i5 14th Gen) Intel Core 5 Series 2 210H - (16 GB/512 GB SSD..."]')

  .click()

  await page1.waitForTimeout(3000)

  await page1.locator('//span[text()="More"]').first().hover() //no control to new opened tab

  let page2= await context.newPage()

  await
page2.goto('https://www.amazon.in/s?k=shirt&crd=2QI7AF3UM35HK&sprefix=shirt%2Caps%2C279&ref=nb_sb_noss_2')

  await page2.waitForTimeout(3000)

  await page2.locator('h2[class="a-size-base-plus a-spacing-none a-color-base a-text-normal"]').first().click()

  await page2.waitForTimeout(3000)

})
```

```
//giving control to the newly opened tab

test.only('giving control to the newly opened tab', async ({browser}) => {

  let context = await browser.newContext()

  let page= await context.newPage()

  await
page.goto('https://www.flipkart.com/search?q=laptop&otracker=search&otracker1=search&marketplace=FLIPKART&as-show=on&as=off')

  await page.waitForTimeout(3000)

  let [newpage] = await Promise.all([

    page.waitForEvent('popup'),

    page.locator('//div[text()="MOTOROLA Motobook 60 Full Metal 2.8K OLED (i5 14th Gen) Intel Core 5 Series 2 210H - (16 GB/512 GB SSD..."]')

    .click()

  ])

  await newpage.locator('//span[text()="More"]').first().hover()

  await newpage.waitForTimeout(3000)

})
```

New Window:

```
//window handles

test.only("window handles", async ({browser}) => {

  let context = await browser.newContext()

  let page = await context.newPage()

  await
page.goto('https://demoapps.qspiders.com/ui/browser/multipleWindow?sublist=2')

  await page.waitForTimeout(2000)

  let [new_window] = await Promise.all([

    page.waitForEvent('popup'),

    page.locator('//button[text()="Shop Now"]').click()

  ])

  await new_window.waitForTimeout(2000)

})
```

```

    })

    await new_window.waitForTimeout(2000)

    await new_window.locator('//button[text()="Add to Cart"]').click()

    await new_window.waitForTimeout(8000)
  })

```

Dialogs

Dialogs in Playwright are browser pop-up messages that interrupt the webpage and require user interaction, such as accepting, dismissing, or entering text.

Playwright provides the **dialog** event to handle these browser dialogs.

Types of Dialogs

1. **Alert** – Displays a message with an **OK** button.
2. **Confirm** – Displays a message with **OK** and **Cancel**.
3. **Prompt** – Displays a message with an input field and **OK/Cancel**.

By default, Playwright automatically dismisses dialogs if there is no **dialog event listener.**

Handling Dialogs

Playwright provides two commonly used event listeners:

1. **page.on()**

Syntax

(if we use async, await is important)

```

page.on("dialog", async (dialog) =>
{
    await dialog.accept();
});

```

(if we don't use await)

```

page.on("dialog", (dialog) =>
{
    dialog.accept();
});

```

Here:

- `"dialog"` → event name
- `dialog` → parameter containing the Dialog object
- `dialog.accept()` → clicks OK
- `page.on()` keeps listening for the event and can handle multiple dialogs.

2. `page.once()`

Syntax

(if we use `async`, `await` is important)

```
page.once("dialog", async (dialog) => {  
  await dialog.accept();  
});
```

(if we don't use `await`)

```
page.once("dialog", (dialog) => {  
  dialog.accept();  
});
```

Here:

- `"dialog"` → event name
- `dialog` → parameter containing the Dialog object
- `dialog.accept()` → clicks OK
- `page.once()` → listens for the event only once and then stops listening

`page.on()`

Listens for the event continuously

Can handle multiple dialogs

Listener remains active after handling the event

Used when the event may occur multiple times

Example: Multiple alerts appearing

`page.once()`

Listens for the event only once

Handles only the first dialog

Listener is automatically removed after the first event

Used when the event is expected to occur only once

Example: One alert appearing

Code:

```
import {test} from "@playwright/test"
//auto handled - auto denied
test("auto denied", async ({page}) => {
  await page.goto('https://testautomationpractice.blogspot.com/')
  await page.waitForTimeout(3000)
  await page.locator('button[id="alertBtn"]').click()
  await page.waitForTimeout(3000)
})

//event listeners
test("page.on", async ({page}) => {
  page.on(("dialog"), async (dialog) => {
    dialog.accept()
  })
  await page.goto('https://testautomationpractice.blogspot.com/')
  await page.waitForTimeout(3000)
  //simple dialog
  await page.locator('button[id="alertBtn"]').click()
  await page.waitForTimeout(3000)
  //confirm dialog
  await page.locator('button[id="confirmBtn"]').click()
  await page.waitForTimeout(3000)
  //fails to handle prompt dialog
  await page.locator('button[id="promptBtn"]').click()
  await page.waitForTimeout(3000)
})

//dealing with prompt
test.only("dealing with all dialogs", async ({page}) => {
  page.on(("dialog"), async (dialog) => {
    if(dialog.type() === "alert")
    {
      dialog.accept()
      console.log(dialog.message());
    }
    else if(dialog.type() === "confirm")
    {
      dialog.accept()
    }
    else if (dialog.type() === "prompt")
    {
      dialog.accept("babrbie")
    }
  })
  await page.goto('https://testautomationpractice.blogspot.com/')
  await page.waitForTimeout(3000)
  //simple dialog
  await page.locator('button[id="alertBtn"]').click()
  await page.waitForTimeout(3000)
```

```

    //confirm dialog
    await page.locator('button[id="confirmBtn"]').click()
    await page.waitForTimeout(3000)
    //prompt dialog
    await page.locator('button[id="promptBtn"]').click()
    await page.waitForTimeout(3000)
  })

  //page.once
  test.only("dealing with all dialogs using page.once", async ({page}) => {
    page.once("dialog", async (dialog) => {
      if(dialog.type()=== "alert")
      {
        dialog.accept()
      }
      else if(dialog.type()=== "confirm")
      {
        dialog.accept()
      }
      else if (dialog.type()=== "prompt")
      {
        dialog.accept("babrbie")
      }
    })
    await page.goto('https://testautomationpractice.blogspot.com/')
    await page.waitForTimeout(3000)
    //simple dialog
    await page.locator('button[id="alertBtn"]').click()
    await page.waitForTimeout(3000)
    //confirm dialog
    await page.locator('button[id="confirmBtn"]').click()
    await page.waitForTimeout(3000)
    //prompt dialog
    await page.locator('button[id="promptBtn"]').click()
    await page.waitForTimeout(3000)
  })

```

Notifications

A notification is a message displayed by the browser or web application to inform the user about an event or activity.

By default, notifications are **denied** in Playwright.

Types of Notifications

1. Permission Popup

A **permission popup** is a message displayed by the browser asking the user for permission before a website can access a particular feature.

Example: Allowing a website to send notifications.

2. Web Push Notification

A **web push notification** is the actual message sent by a website to the user **after** notification permission has been granted.

Example: "You have a new message" or "Your order has been shipped."

Code:

```
import {test} from "@playwright/test"

//auto handling of notifications -- denied
test("notifications", async ({browser}) => {
    let context = await browser.newContext()
    let page = await context.newPage()
    await
page.goto('https://demoapps.qspiders.com/ui/browserNot?sublist=0&scenario=1')
    await page.waitForTimeout(3000)
    await page.locator('button[id="browNotButton"]').click()
    await page.waitForTimeout(3000)
    let state = await page.evaluate(()=>{
        return Notification.requestPermission()
    })
    console.log(state);
})

//accepting the notification --- granted
test.only("handling notifications", async ({browser}) => {
    let context = await browser.newContext({permissions:["notifications"]})
    let page = await context.newPage()
    await
page.goto('https://demoapps.qspiders.com/ui/browserNot?sublist=0&scenario=1')
    await page.waitForTimeout(3000)
    await page.locator('button[id="browNotButton"]').click()
    await page.waitForTimeout(3000)
    let state = await page.evaluate(()=>{
        return Notification.requestPermission()
    })
    console.log(state);
})
```

```
}}
```

Iframes

An **iframe (Inline Frame)** is an HTML element used to **embed another webpage or HTML document inside the current webpage**.

In Playwright, elements inside an iframe **cannot always be accessed directly from the main page**. We need to switch to the iframe context first.

How to handle Iframes?

1. `frameLocator()`

`frameLocator()` is used to locate and interact with elements inside an iframe

Syntax: `await page.frameLocator('locator')`

2. `frame()`

`frame()` is used to access a specific iframe (Frame object) so that we can interact with elements inside that frame.

Syntax: `await page.frame({url: 'iframe_url'})`

`await page.frame({name: 'name_value'})`

Note: `page.frame()` → Used to access a frame using its **name** or **url**.

Code:

```
import {test} from "@playwright/test"
//total no of iframes
test("total iframes", async ({page}) => {
    await page.goto('https://ui.vision/demo/webtest/frames/')
    let totalframes = await page.frames()
    console.log(totalframes.length);
})
//title of all the frames
test("title of all frames", async ({page}) => {
    await page.goto('https://ui.vision/demo/webtest/frames/')
    let totalframes = await page.frames()
    for(let frame of totalframes)
```

```

    {
        console.log(await frame.title());
        await page.waitForTimeout(3000)
    }
}))
//switching frames
test.only("switching frames", async ({page}) => {
    await page.goto('https://ui.vision/demo/webtest/frames/')
    await page.waitForTimeout(3000)
    //frame()
    let frame1 = await page.frame({url:
'https://ui.vision/demo/webtest/frames/frame_1'})
    // await page.frame({name : 'firstFr'})
    await page.waitForTimeout(3000)
    await frame1.locator('input[name="mytext1"]').fill("shinchan")
    await page.waitForTimeout(3000)
    //frameLocator()
    let frame2 = await page.frameLocator('frame[src="frame_2.html"]')
    await page.waitForTimeout(3000)
    await frame2.locator('input[name="mytext2"]').fill('nohara')
    await page.waitForTimeout(3000)
    //frame()
    let frame3 = await
page.frame({url:'https://ui.vision/demo/webtest/frames/frame_3'})
    await page.waitForTimeout(3000)
    await frame3.locator('input[name="mytext3"]').fill('please fill in the form')
    await page.waitForTimeout(3000)
}))

```

File Upload

File Upload is the process of uploading a file from the local system to a web application using a file input element.

Method Used for File Upload

Playwright provides the `setInputFiles()` method to upload files.

`setInputFiles()` is a Playwright method used to set one or more files on an `<input type="file">` element.

Syntax:

```
await page.locator('input[type="file"]').setInputFiles('file-path');
```

Code:

```
import {test} from "@playwright/test"
import path from "node:path"

//uploading single file from project file
test("uploading single file from project file", async ({page}) => {
    await page.goto('https://testautomationpractice.blogspot.com/')
    await
page.locator('input[id="singleFileInput"]').setInputFiles("C:/Users/User/OneDrive/Desktop/playwright_B1/Demo files/demo1")
    await page.waitForTimeout(3000)
    await page.getByRole("button", {name:"Upload Single File"}).click()
    await page.waitForTimeout(3000)
})

//uploading multiple files from the project file
test("uploading multiple files from the project file", async ({page}) => {
    await page.goto('https://testautomationpractice.blogspot.com/')
    await page.waitForTimeout(3000)
    await
page.locator('input[id="multipleFilesInput"]').setInputFiles(["C:/Users/User/OneDrive/Desktop/playwright_B1/Demo files/demo1" ,
    "C:/Users/User/OneDrive/Desktop/playwright_B1/Demo files/demo2" ])
    await page.waitForTimeout(3000)
    await page.getByText("Upload Multiple Files").click()
    await page.waitForTimeout(3000)
})

//printing the directory of current control
test("printing the directory of current control", async ({page}) => {
    await page.goto('https://testautomationpractice.blogspot.com/')
    console.log(__dirname);
})
```

```
//traversing from current directory
test("traversing from current directory", async ({ page }) => {
  await page.goto('https://testautomationpractice.blogspot.com/');
  let filePath = path.join(__dirname, "../Demo files/demol.pdf");
  console.log("File Path:", filePath);
  await page.locator('input[id="singleFileInput"]').setInputFiles(filePath);
  await page.waitForTimeout(3000);
});
```

File Download

File downloading is the process of downloading a file from a web application and saving it to the local system.

Method Used

Playwright uses the `page.waitForEvent('download')` method to handle file downloads.

Syntax

```
let [download] = await Promise.all([
  page.waitForEvent('download'),
  page.locator('download-button').click()
]);
```

```
await download.saveAs('file-path');
```

Code:

```
import {test} from "@playwright/test"
import path from "node:path"
//downloading a small size file
test("downloading a small size file", async ({page}) => {
  await page.goto("https://demoapps.qspiders.com/ui/download?sublist=0")
  await page.waitForTimeout(2000)
  await page.getByPlaceholder("Enter text here").fill("hello")
  await page.waitForTimeout(2000)
  await page.locator('button[id="downloadButton"]').click()
  await page.waitForTimeout(3000)
})

//download the file using promise.all()
test("download the file using promise.all", async ({page}) => {
  await page.goto("https://demoapps.qspiders.com/ui/download?sublist=0")
  await page.waitForTimeout(2000)
  await page.getByPlaceholder("Enter text here").fill("hello")
  await page.waitForTimeout(2000)
  let [downloadall]= await Promise.all([
    page.waitForEvent("download"),
    page.locator('button[id="downloadButton"]').click()
  ])
})
```

```

    ])
    await page.waitForTimeout(3000)
  })

  //giving the loc to save the downloaded file
  test("giving the loc to save the downloaded file", async ({page}) => {
    await page.goto("https://demoapps.qspiders.com/ui/download?sublist=0")
    await page.waitForTimeout(2000)
    await page.getByPlaceholder("Enter text here").fill("hello all!! how are you??")
    await page.waitForTimeout(2000)
    let [downloadall]= await Promise.all([
      page.waitForEvent("download"),
      page.locator('button[id="downloadButton"]').click()
    ])
    await page.waitForTimeout(3000)
    //saveas
    await
downloadall.saveAs("C:/Users/User/OneDrive/Desktop/playwright_B1/Downloads/greet.txt")
  })

  //downloading file with the suggested filename
  test("downloading file with the suggested filename", async ({page}) => {
    await page.goto("https://demoapps.qspiders.com/ui/download?sublist=0")
    await page.waitForTimeout(2000)
    await page.getByPlaceholder("Enter text here").fill("hello all!! how are you??")
    await page.waitForTimeout(2000)
    let [downloadall]= await Promise.all([
      page.waitForEvent("download"),
      page.locator('button[id="downloadButton"]').click()
    ])
    await page.waitForTimeout(3000)
    let filename = await downloadall.suggestedFilename()
    console.log(filename);
    console.log(await downloadall.path());

    //deleting the downloaded file
    await downloadall.delete()
  })

  //saving to particular location with suggested name
  test.only("saving to particular location with suggested name", async ({page}) => {
    await page.goto("https://demoapps.qspiders.com/ui/download?sublist=0")
    await page.waitForTimeout(2000)
    await page.getByPlaceholder("Enter text here").fill("hello all!! how are you??")
    await page.waitForTimeout(2000)
  })

```

```
let [downloadall]= await Promise.all([
    page.waitForEvent("download"),
    page.locator('button[id="downloadButton"]').click()
])
await page.waitForTimeout(3000)
let suggestedFileName=await downloadall.suggestedFilename();
let downloadpath = path.join(__dirname,"../Downloads/"+suggestedFileName);
await downloadall.saveAs(downloadpath)
})
```

Data-Driven Testing (DDT)

Data-Driven Testing is a testing approach where the same test case is executed multiple times using different sets of test data.

Instead of writing separate test cases for each data set, we write one test case and provide different data from an external source such as an Excel file, CSV file, JSON file, or database.

Why do we need Data-Driven Testing?

- To avoid writing the same test case multiple times.
- To test the application with different sets of data.
- To reduce code duplication.
- To make test scripts easier to maintain.
- To increase test coverage.
- To separate test data from test logic.
- If the test data changes, we can update the data source without changing the test script.

Simple Example

Suppose we want to test login with 5 different users.

Without DDT:

Test 1 → username1 + password1
Test 2 → username2 + password2
Test 3 → username3 + password3
Test 4 → username4 + password4
Test 5 → username5 + password5

With DDT:

One Login Test



Read different data



Execute the same test multiple times

So, Data-Driven Testing = One test case + Multiple sets of test data.

1. DDT from Excel files:

Open your playwright project folder → create a DDT folder inside it → inside DDT folder create one excel file → now open your playwright project folder in VS Code

To use excel libraries, install Excel package:

```
npm install exceljs
```

Why do we use it in Playwright?

In **Data-Driven Testing**, we may store test data in an Excel file. `exceljs` allows our Playwright script to interact with that Excel file.

Code:

```
import {test} from "@playwright/test"

import excel from "exceljs"

import path from "node:path"

//fetching cell object

test("fetching cell object",async ({page}) => {

    let book = await new excel.Workbook()

    await book.xlsx.readFile(path.join(__dirname, "../Data Driven
Testing/singledata.xlsx"))

    let sheet = await book.getWorksheet("Sheet1")

    let data = await sheet.getRow(1).getCell(1)

    console.log(data);

})

//fetching single data

test.only("fetching single data",async ({page}) => {
```

```

    let book = await new excel.Workbook()

    await book.xlsx.readFile(path.join(__dirname, "../Data Driven
Testing/singledata.xlsx"))

    let sheet = await book.getWorksheet("Sheet1")

    //let data = await sheet.getRow(1).getCell(1).value

    let data = await sheet.getRow(1).getCell(1).toString()

    console.log(data);

  })

  //fetching multiple data- keeping row constant

import {test} from "@playwright/test"

import excel from "exceljs"

import path from "node:path"

test("fetching multiple data", async ({page}) => {

    let book = await new excel.Workbook()

    await book.xlsx.readFile(path.join(__dirname, "../Data Driven
Testing/multipledata.xlsx"))

    let sheet = await book.getWorksheet("Sheet1")

    for ( let i = 1 ; i<=4 ; i++)

    {

        let data = await sheet.getRow(1).getCell(i).toString()

        console.log(data);

    }

  })

  //using data in test stript

import {test} from "@playwright/test"

import excel from "exceljs"

import path from "node:path"

test("using test data in script", async ({page}) => {

```

```

    let book = new excel.Workbook()

    await book.xlsx.readFile(path.join(__dirname, "../Data Driven
Testing/SSLoginexcel.xlsx"))

    let sheet = await book.getWorksheet("Sheet1")

    let alldata = []

    for (let dataset = 1; dataset <=sheet.actualRowCount ; dataset++)
    {

        let row = sheet.getRow(dataset)

        let url = row.getCell(1).toString()

        let usn = row.getCell(2).toString()

        let pwd = row.getCell(3).toString()

        alldata.push({url : url , username : usn , password : pwd})

    }

    console.log(alldata);

    for (let dummy of alldata)
    {

        await page.goto(dummy.url)

        await page.locator('button[id="loginBtn"]').click()

        await page.getByRole("textbox" , {name:"Email"}).fill(dummy.username)

        await page.getByRole("textbox", {name:"Password"}).fill(dummy.password)

        await page.locator('button[id="Login"]').click()

    }

    await page.waitForTimeout(3000)
}))

//writing data into excel

test.only("writing data into excel", async ({page}) => {

    let book = new excel.Workbook()

```

```

    await book.xlsx.readFile(path.join(__dirname, "../Data Driven
Testing/multipledata.xlsx"))

    let sheet = book.getWorksheet("Sheet2")

    if (!sheet)

    {

        sheet = book.addWorksheet("Sheet2")

    }

    sheet.getRow(1).getCell(1).value = "hello"

    await book.xlsx.writeFile(path.join(__dirname, "../Data Driven
Testing/multipledata.xlsx"))

    console.log("data written successfully");

})

```

2. DDT from JSON files:

In Playwright, you can perform **Data-Driven Testing (DDT)** by keeping test data in a JSON file and running the same test for multiple sets of data.

read.json

```

{
  "greet" : "hello",
  "name" : "shinchan"
}

```

iterate.json

```

[
  {
    "greet" : "hello"
  },
  {
    "greet" : "hi"
  },
  {
    "greet" : "bye"
  }
]

```

Code:

```
import {test} from "@playwright/test"
import fs from "fs"
import readdata from "../Data Driven Testing/read.json"
import iteratedata from "../Data Driven Testing/iterate.json"
//reading the data from the json
test("reading the data from the json", async () => {
    let datafile =
fs.readFileSync("C:/Users/User/OneDrive/Desktop/playwright_B1/Data
Driven Testing/read.json")
    let data = JSON.parse(datafile)
    console.log(data.greet);
    console.log(data.name);
})
//through importing data file to script
test("importing test data file", async () => {
    let greetdata = readdata.greet
    let namedata = readdata.name
    console.log(greetdata);
    console.log(namedata);
})

//iterating over json array
test("iterating over json array", async () => {
    let datafile1 =
fs.readFileSync("C:/Users/User/OneDrive/Desktop/playwright_B1/Data
Driven Testing/iterate.json")
    let data1 = JSON.parse(datafile1)
    data1.forEach(Element=>{
        console.log(Element.greet);
    })
})
//through importing data file to script
test("iterating", async () => {
    for (let g of iteratedata)
    {
        console.log(g.greet);
    }
})
```

Utilizing the test data file into script:

SSLogin.json:

```
{
  "url" : "https://shoppersstack.com/",
  "username" : "supriyasharmaa1208@gmail.com",
  "password" : "Password@123"
}
```

Code:

```
import {test} from "@playwright/test"
import logindata from "../Data Driven Testing/SSLogin.json"

test("using json data into login file",async ({page}) => {
  let url = logindata.url
  let usn = logindata.username
  let pwd = logindata.password
  await page.goto(url)
  await page.waitForTimeout(2000)
  await page.locator('button[id="loginBtn"]').click()
  await page.locator('input[id="Email"]').fill(usn)
  await page.locator('input[id="Password"]').fill(pwd)
  await page.waitForTimeout(2000)
})
```

Dealing with multiple credentials in test data file:

TALogin.json

```
[{
  "url" : "https://practicetestautomation.com/practice-test-login/",
  "username" : "student",
  "password" : "Password123"
},
{
  "url" : "https://practicetestautomation.com/practice-test-login/",
  "username" : "studenet",
  "password" : "Passwoxrd123"
},
{
  "url" : "https://practicetestautomation.com/practice-test-login/",
  "username" : "steudent",
  "password" : "Pasdsword123"
}]
```

Code:

```
import {test} from "@playwright/test"
import logindata from "../Data Driven Testing/TALogin.json"
//dealing with multiple credentials
test("dealing with multiple credentials", async ({page}) => {
  for (let data of logindata)
  {
    let url = data.url
    let usn = data.username
    let pwd = data.password
    await page.goto(url)
    await page.waitForTimeout(2000)
    await page.locator('input[id="username"]').fill(usn)
    await page.waitForTimeout(2000)
    await page.locator('input[id="password"]').fill(pwd)
    await page.waitForTimeout(2000)
  }
})
```

POM (Page Object Model)

POM (Page Object Model) is a design pattern used in Playwright automation to organize locators and actions of a webpage into separate classes/files.

POM – Important Use Cases:

1. **Reusability** – Reuse common page actions.
2. **Maintainability** – Easy to update locators.
3. **Less Code Duplication** – Avoid repeated code.
4. **Easy Debugging** – Locate and fix issues easily.
5. **Better Readability** – Tests are easier to understand.
6. **Separation of Code** – Test logic and page actions are separated.
7. **Easy Locator Management** – Keep page locators organized.
8. **Scalability** – Suitable for large automation projects.

POM:

```
class testlogin
{
  constructor(page)
  {
```

```

        this.usernameTF = page.locator('input[id="username"]')

        this.passwordTF = page.locator('input[id="password"]')

        this.submitBtn = page.locator('button[id="submit"]')

    }

}

export default testlogin

```

TATLogin.json:

```

{
  "url" : "https://practicetestautomation.com/practice-test-login/",
  "username" : "student",
  "password" : "Password123"
}

```

Code:

```

import {test} from "@playwright/test"
import testlogin from "../POM/Test_login.js"
import logindata from "../Data Driven Testing/TALogin.json"
//only with pom
test("POM" , async ({page}) => {
    let loginobj = new testlogin(page)
    await
page.goto("https://practicetestautomation.com/practice-test-login/")
    await page.waitForTimeout(3000)
    await loginobj.usernameTF.fill("student")
    await page.waitForTimeout(3000)
    await loginobj.passwordTF.fill("Password123")
    await page.waitForTimeout(3000)
    await loginobj.submitBtn.click()
    await page.waitForTimeout(3000)
})

//with pom and ddt
test.only("with pom and ddt", async ({page}) => {
    let loginobj1 =new testlogin(page)
    let url = logindata.url
    let usn = logindata.username
    let pwd = logindata.password
    await page.goto(url)
    await page.waitForTimeout(3000)

```

```
    await loginobj1.usernameutf.fill(usn)
    await page.waitForTimeout(3000)
    await loginobj1.passwordtf.fill(pwd)
    await page.waitForTimeout(3000)
    await loginobj1.submitbtn.click()
    await page.waitForTimeout(3000)
  })
```